
REPORT TECNICO

Machine Learning Applicato

Problema B: Regressione

Life Expectancy (WHO)

Kaggle Dataset — Organizzazione Mondiale della Sanità

Gruppo: CereaMorlotti

Autori: Morlotti Andrea
Cerea Gianluca

Data: 30 giugno 2026

Piattaforma: Microsoft Fabric · Power BI · Power Query

Dataset: kaggle.com/datasets/kumarajarshi/life-expectancy-who

Contents

1	Introduzione e motivazione della scelta del dataset	2
1.1	Dataset scelto	2
1.2	Adeguatezza al problema di regressione	2
1.3	Variabile target	2
1.4	Contesto applicativo	2
2	Analisi Esplorativa dei Dati (EDA)	2
2.1	Distribuzione della variabile target	2
2.2	Analisi della variabile categoriale status	3
2.3	Andamento temporale	3
2.4	Analisi dei valori mancanti	3
2.4.1	Pattern di missingness	4
2.5	Analisi degli outlier	5
2.6	Correlazione con il target	5
3	Pipeline di Data Cleaning e Feature Engineering	5
3.1	Workflow pre-cleaning	6
3.2	Pipeline Power Query	7
3.3	Risultati del cleaning	7
3.4	Strategia di imputazione — scelta metodologica	8
4	Descrizione e confronto dei modelli sviluppati	9
4.1	Preprocessing per i modelli	9
4.2	Modelli sviluppati	10
4.3	Ottimizzazione iperparametri	10
4.4	AutoML con FLAML	10
4.5	Deep Learning con Keras	11
5	Analisi delle metriche e scelta del modello migliore	11
5.1	Classifica finale — tutti i modelli	11
5.2	Modello scelto	11
5.3	Interpretazione per famiglia di modelli	12
5.4	Feature Importance	12
5.5	Analisi dei residui	12
5.6	Analisi overfitting	13
6	Interpretazione dei risultati e conclusioni	14
6.1	Principali risultati	14
6.2	Limitazioni	14
6.3	Possibili sviluppi futuri	14
7	Divisione del lavoro tra i membri del gruppo	15
8	Link al Workspace Fabric	15

1. Introduzione e motivazione della scelta del dataset

1.1. Dataset scelto

Il dataset utilizzato è **Life Expectancy (WHO)**, prodotto dall'Organizzazione Mondiale della Sanità (OMS) e disponibile su Kaggle. Contiene dati annuali per 193 paesi nel periodo 2000–2015, per un totale di **2938 osservazioni** e **22 variabili** che coprono indicatori sanitari, economici e demografici.

1.2. Adeguatezza al problema di regressione

Table 1: Verifica dei requisiti minimi della consegna

Requisito	Valore	Esito
Dimensione minima (500 righe)	2938 righe	✓
Feature predittive (≥ 5)	19 usate nei modelli	✓
Mix numeriche e categoriali	Numeriche + <code>status</code> (binaria)	✓
Valori mancanti presenti	Sì: <code>population</code> 22%, <code>hepatitis_b</code> 19%	✓
Sorgente pubblica citabile	Kaggle / OMS, licenza Open	✓

1.3. Variabile target

La variabile target è `life_expectancy`, un valore numerico continuo che rappresenta l'aspettativa di vita alla nascita in anni. Il range osservato va da **36.3 a 89.0 anni**, con distribuzione leggermente asimmetrica verso sinistra.

1.4. Contesto applicativo

Predire l'aspettativa di vita a partire da indicatori sanitari ed economici permette di capire quali fattori incidono di più sulla salute di una popolazione. Questo tipo di analisi può essere utile per orientare scelte e priorità, soprattutto nei paesi in via di sviluppo.

2. Analisi Esplorativa dei Dati (EDA)

Nota metodologica

L'EDA è stata condotta esclusivamente sul **training set raw** (dopo lo split 80/20), per evitare qualsiasi data leakage dal test set. I risultati hanno guidato le decisioni di cleaning applicate successivamente in Power Query.

2.1. Distribuzione della variabile target

La distribuzione di `life_expectancy` sul training set presenta una forma leggermente asimmetrica verso sinistra, con concentrazione dei valori nell'intervallo 70–80 anni e una coda verso valori bassi (paesi con aspettativa di vita ridotta).

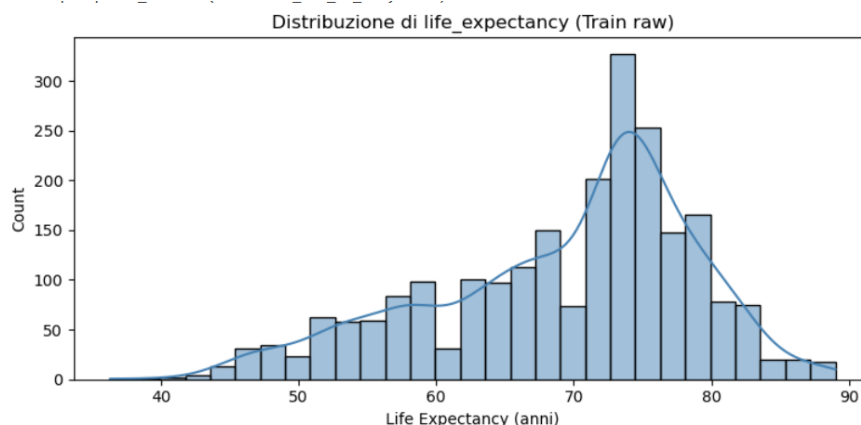


Figure 1: Distribuzione della variabile target `life_expectancy` sul training set raw

2.2. Analisi della variabile categoriale status

La variabile `status` presenta un forte squilibrio: i paesi *Developing* sono nettamente più numerosi dei *Developed*. Questo squilibrio motiva la scelta di calcolare le mediane di imputazione separatamente per gruppo.

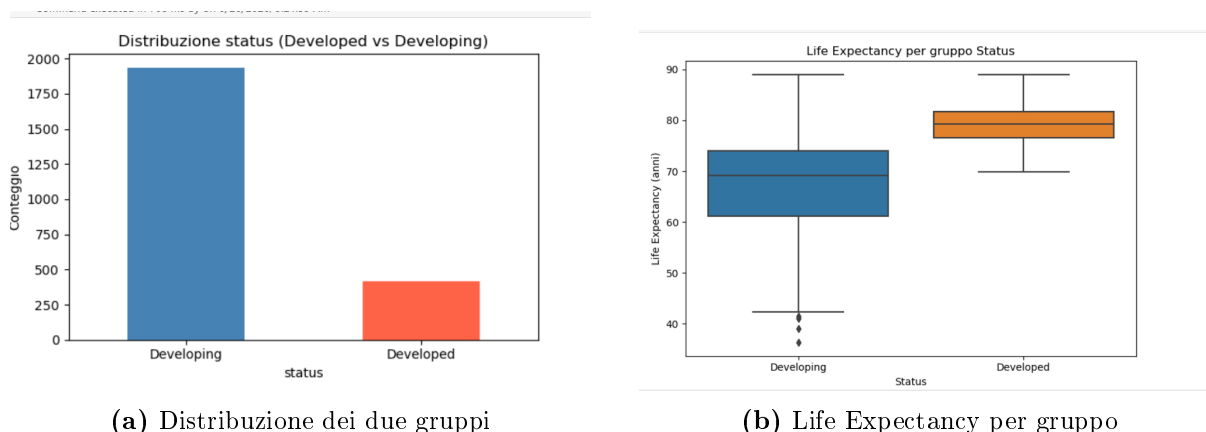


Figure 2: Analisi della variabile `status`: distribuzione e relazione con il target

Il boxplot evidenzia una differenza marcata: i paesi *Developed* hanno un'aspettativa di vita mediana di circa 79 anni, contro i circa 69 anni dei *Developing*. Questa differenza conferma che `status` è una variabile predittiva rilevante, preservata nel modello tramite label encoding.

2.3. Andamento temporale

L'aspettativa di vita media mostra un trend crescente nel periodo 2000–2015, passando da circa 67 a oltre 72 anni. Questo conferma la coerenza temporale del dataset e il progressivo miglioramento delle condizioni sanitarie globali.

2.4. Analisi dei valori mancanti

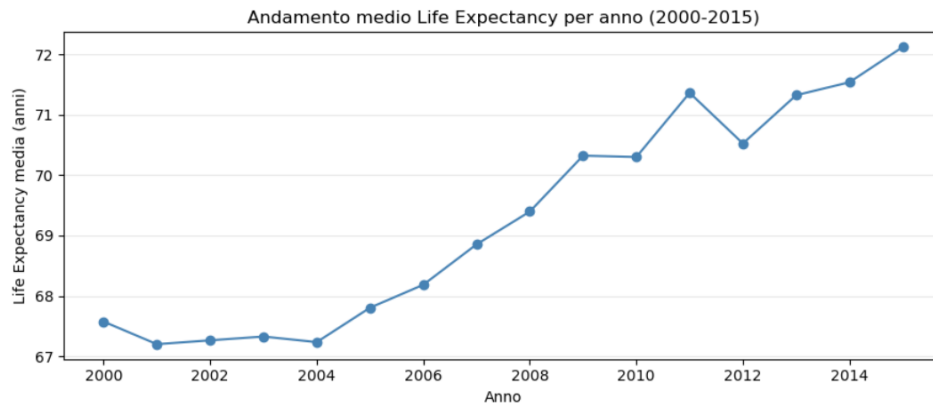


Figure 3: Andamento medio dell’aspettativa di vita per anno (2000–2015)

Table 2: Valori mancanti per colonna (training set raw)

Colonna	Missing %	Strategia
population	22.19%	Rimossa ($r=0.02$ con target)
hepatitis_b	18.82%	Mediana per gruppo status
gdp	15.25%	Mediana per gruppo status
total_expenditure	7.69%	Mediana per gruppo status
alcohol	6.60%	Mediana per gruppo status
income_composition	5.68%	Mediana per gruppo status
schooling	5.55%	Mediana per gruppo status

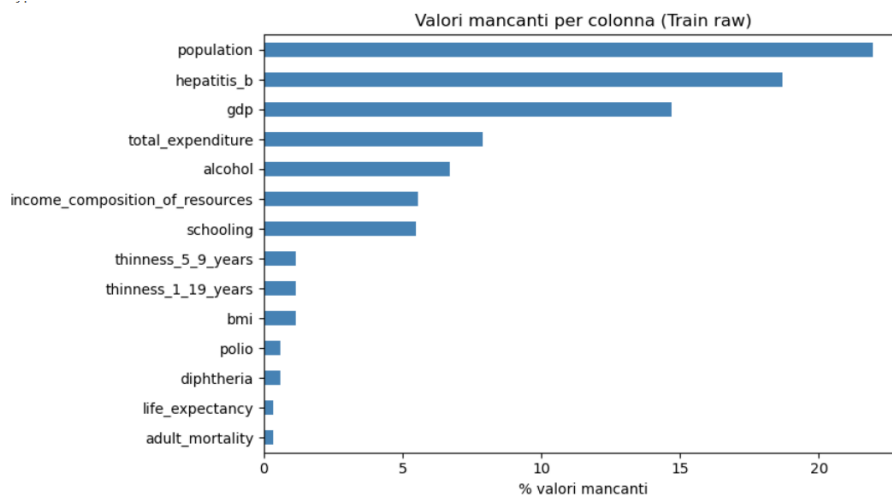


Figure 4: Percentuale di valori mancanti per colonna (training set raw)

2.4.1. Pattern di missingness

I valori mancanti non sono distribuiti in modo casuale: sono molto più frequenti nei paesi *Developing* rispetto ai *Developed*. In altre parole, la presenza di un valore mancante dipende dal gruppo **status** a cui appartiene il paese. Per questo motivo l’imputazione è stata fatta usando la mediana calcolata separatamente per i due gruppi, e non un’unica mediana globale: così ogni valore mancante viene sostituito con un valore più rappresentativo del contesto del paese.

2.5. Analisi degli outlier

Inizialmente è stata fatta un'analisi rimuovendo gli outlier con metodo IQR, per verificare se questo potesse migliorare le performance dei modelli. Riguardando i dati con più attenzione, però, ci si è accorti che proprio in quei valori estremi era contenuta informazione importante. La tabella seguente mostra il confronto:

Table 3: Impatto della rimozione outlier sulle metriche (R^2 test)

Modello	R^2 con rimozione	R^2 senza rimozione	ΔR^2
RandomForest Tuned	0.957	0.970	+0.013
GradientBoosting Tuned	0.955	0.969	+0.014
LinearRegression	0.787	0.827	+0.040
Keras DL	-81.7	0.875	+82.6

I valori estremi in variabili come `hiv_aids`, `measles` e `under_five_deaths` non sono errori di misura ma rappresentano situazioni reali: paesi colpiti da epidemie o con condizioni sanitarie particolarmente critiche. Rimuovere quei dati significava eliminare proprio i casi più significativi del dataset. Per questo la pipeline finale non applica nessuna rimozione di outlier.

Il valore $R^2 = -81.7$ della rete neurale con rimozione outlier non indica un semplice peggioramento ma un vero e proprio collasso del modello: rimuovendo gli outlier il training set si è ridotto e sbilanciato al punto che la rete non è riuscita a convergere, producendo predizioni molto distanti dai valori reali. Un R^2 fortemente negativo significa infatti che il modello predice peggio di una semplice media costante. Questo conferma quanto quei dati fossero importanti per l'apprendimento.

2.6. Correlazione con il target

Le feature più correlate con `life_expectancy` nel training set sono `schooling` e `income_composition_o` (positive), `adult_mortality` e `hiv_aids` (negative).

Gli scatter plot delle quattro feature più correlate confermano le relazioni: `schooling` e `income_composition` mostrano una relazione positiva chiara, mentre `adult_mortality` ha una relazione negativa marcata. La colorazione per `status` evidenzia come i paesi *Developed* si concentrino nelle zone ad alta aspettativa di vita.

Nota metodologica

Multicollinearità: `thinness_1_19_years` e `thinness_5_9_years` presentano correlazione reciproca ≈ 0.93 e sono quasi ridondanti. I modelli ensemble gestiscono la multicollinearità nativamente; per i modelli lineari sarebbe utile rimuovere una delle due.

3. Pipeline di Data Cleaning e Feature Engineering

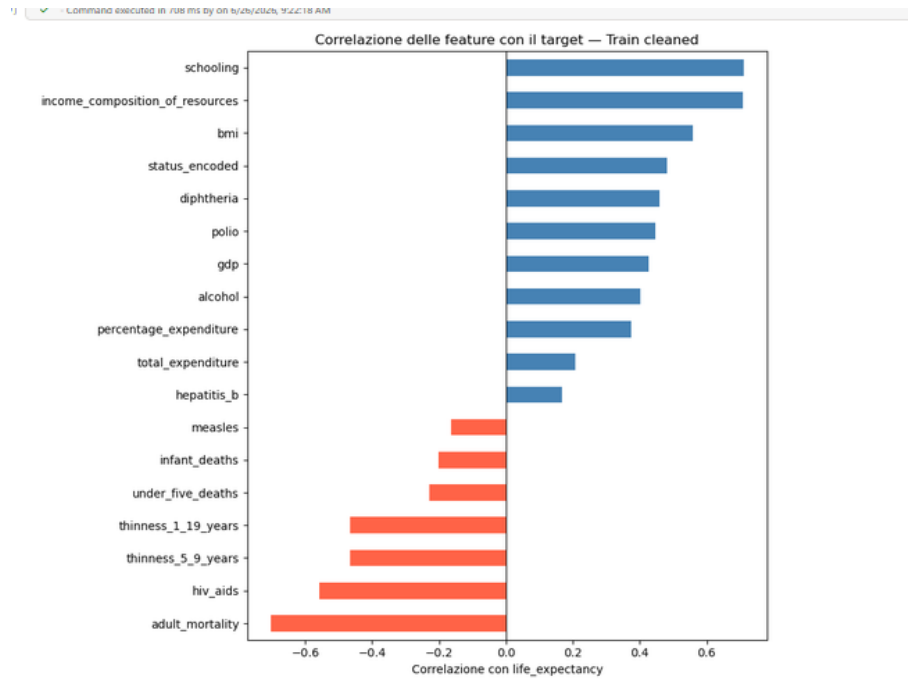


Figure 5: Correlazione delle feature con il target (training set cleaned)

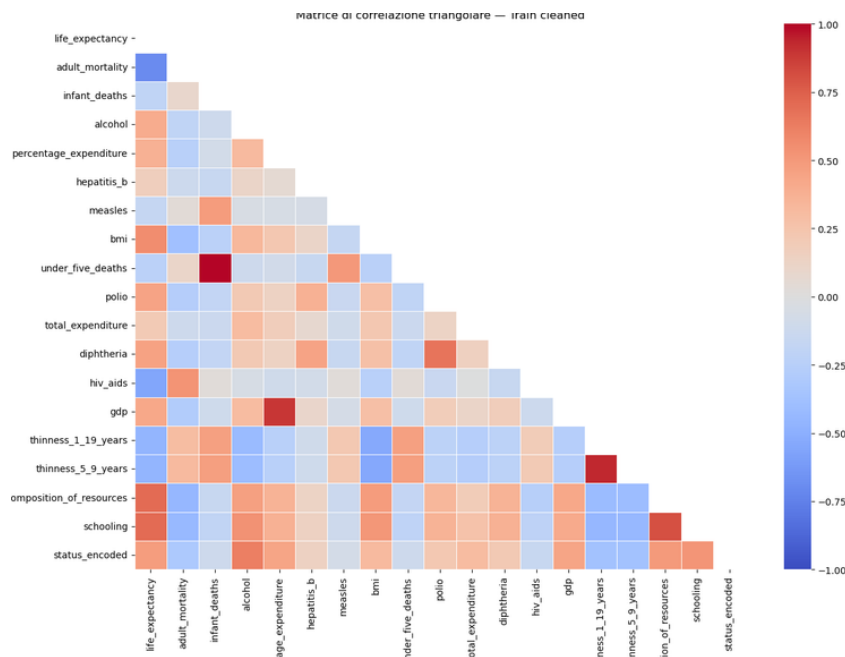


Figure 6: Matrice di correlazione triangolare tra le variabili (training set cleaned)

3.1. Workflow pre-cleaning

Il dataset raw è stato diviso in training set (80%) e test set (20%) **prima** di qualsiasi operazione di cleaning, con seed fisso `random_state=42` per garantire la riproducibilità. Lo split è stato eseguito in Python anziché in Power Query perché quest'ultimo non permette di impostare un seed fisso.

- **Training set raw:** 2350 righe (80%)

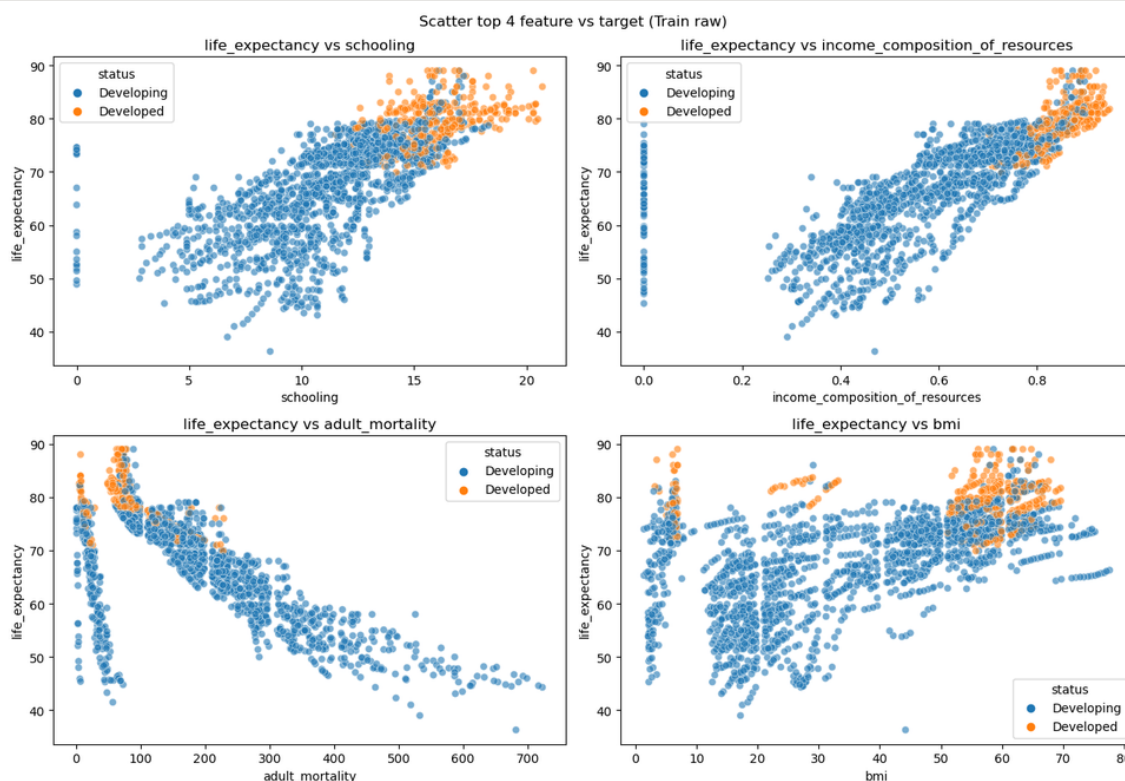


Figure 7: Scatter plot delle quattro feature più correlate con il target, colorate per **status**

- **Test set raw:** 588 righe (20%)

Perché splittare prima del cleaning: la divisione tra train e test è stata fatta come primissima operazione, prima di qualsiasi pulizia o imputazione. In questo modo il test set rimane completamente “sconosciuto” fino alla valutazione finale: tutte le decisioni di preprocessing (quali valori imputare, con quali mediane) vengono prese guardando solo i dati di training. Se avessimo pulito tutto il dataset insieme e poi splittato, le informazioni del test set avrebbero influenzato il cleaning, falsando la valutazione delle performance.

Perché 80/20: è una proporzione standard che bilancia due esigenze opposte: avere abbastanza dati per addestrare bene i modelli (80%) e allo stesso tempo un test set sufficientemente ampio (588 righe) da rendere affidabile la stima delle performance.

3.2. Pipeline Power Query

Il cleaning è stato eseguito interamente in **Fabric Dataflow Gen2 (Power Query)**, applicato separatamente a training set e test set con la stessa pipeline:

3.3. Risultati del cleaning

Dopo l'applicazione della pipeline, entrambe le partizioni risultano prive di valori mancanti e con tipi numerici corretti. La dimensione si riduce di poche righe (eliminazione di duplicati e target nulli) e di una colonna: viene rimossa **population**, mentre **status** viene sostituita da **status_encoded** (trasformazione 1-a-1 che non cambia il numero di colonne). La colonna **country** resta nel dataset salvato ma viene esclusa a livello di modello (vedi Sezione 4.1).

Table 4: Pipeline di cleaning in Power Query

#	Operazione	Motivazione
1	Correzione tipi (locale en-US)	I dati arrivano come stringa dal Delta
2	Rimozione duplicati	Qualità del dato
3	Eliminazione righe target nullo	Non si imputa la variabile target
4	Rimozione population	22% mancante e quasi nessuna correlazione ($r=0.02$)
5	Calcolo mediane su train_raw (Python)	Solo sul train — zero leakage
6	Salvataggio mediane_imputazione nel Lakehouse	Parametri fissi accessibili da Power Query
7	Caricamento mediane da Lakehouse (Power Query)	Stesse mediane per train e test
8	Join con tabella mediane	Requisito di merge/join richiesto
9	Imputazione null con mediane del train	Preserva differenze Developed/Developing
10	Label encoding status	Developed=1, Developing=0
11	Nessuna rimozione outlier	Outlier = variabilità reale

La colonna **population** è stata rimossa per due motivi che insieme la rendevano poco utile: aveva una percentuale di valori mancanti molto alta (22%) e una correlazione praticamente nulla con l'aspettativa di vita ($r=0.02$). Imputare così tanti valori mancanti per una variabile che non aiuta nella previsione avrebbe solo aggiunto rumore al modello.

Table 5: Dimensioni del dataset prima e dopo il cleaning

	Righe	Colonne	Missing	Outlier rimossi
Training raw	2350	22	Sì	—
Training cleaned	2342	21	0	0
Test raw	588	22	Sì	—
Test cleaned	586	21	0	0

3.4. Strategia di imputazione — scelta metodologica

La strategia di imputazione adottata garantisce **zero data leakage** tra training set e test set grazie a un approccio ibrido Python + Power Query:

1. **Python (notebook EDA):** le mediane vengono calcolate esclusivamente sul **train_raw**, separatamente per gruppo **status**. I valori calcolati vengono salvati come tabella **mediane_imputazione** nel Lakehouse.
2. **Power Query (Dataflow):** entrambe le query (train e test) caricano **mediane_imputazione** dal Lakehouse tramite **Table.Pivot** e **NestedJoin**. Il test set viene imputato con

le **stesse mediane del training set**, non con le proprie.

Perché la mediana e non la media: per le variabili con valori mancanti è stata scelta la mediana invece della media perché molte di queste variabili (`gdp`, `percentage_expenditure`, `hepatitis_b`) hanno distribuzioni fortemente asimmetriche con valori estremi. In questi casi la media verrebbe “tirata” verso i valori estremi e non rappresenterebbe bene il valore tipico, mentre la mediana resta stabile e fornisce una stima più realistica.

Perché separare il calcolo dall’applicazione: Power Query non permette di passare parametri calcolati da una query all’altra. La soluzione ingenua — calcolare le mediane direttamente in Power Query su ogni partizione — avrebbe imputato il test set con le proprie mediane, un data leakage che invalida la valutazione del modello. La soluzione adottata separa il calcolo (Python, solo sul train) dall’applicazione (Power Query, su entrambe le partizioni), replicando il comportamento corretto di un `SimpleImputer` scikit-learn fittato sul solo training set.

Verifica empirica: dopo il cleaning, entrambe le partizioni presentano 0 valori mancanti. Il confronto tra mediane salvate e mediane effettive nel dataset conferma la corretta applicazione dei parametri.

4. Descrizione e confronto dei modelli sviluppati

4.1. Preprocessing per i modelli

- **Feature escluse:** `country` (193 valori unici, alta cardinalità)
- **Scaling:** `StandardScaler` fittato solo su `X_train`, applicato a `X_test` con `.transform()`
- **Feature usate:** 19 variabili numeriche incluso `status_encoded`

Perché escludere `country`: la colonna `country` contiene 193 valori diversi, uno per ogni paese. Usarla direttamente avrebbe richiesto di trasformarla in 193 colonne separate (one-hot encoding), aumentando enormemente la complessità del modello senza reale beneficio: le caratteristiche di ogni paese sono già descritte dalle altre variabili (`PIL`, istruzione, indicatori sanitari). Il paese in sé non aggiunge informazione utile alla previsione.

Perché `StandardScaler`: le variabili hanno scale molto diverse tra loro (il `PIL` si misura in migliaia, il consumo di alcol in unità). Alcuni modelli, in particolare quelli lineari e la rete neurale, sono sensibili a queste differenze di scala e darebbero più peso alle variabili con numeri più grandi. Lo `StandardScaler` riporta tutte le variabili sulla stessa scala (media 0 e deviazione standard 1), mettendole sullo stesso piano. Lo scaler viene calcolato solo sul training set e poi applicato al test, per non far “trapelare” informazioni del test durante il preprocessing.

Perché il label encoding per `status`: la variabile `status` ha solo due valori (`Developed/Developing`). In questo caso basta trasformarla in 0 e 1 (label encoding), senza bisogno di tecniche più complesse: con una variabile binaria il label encoding non introduce nessun ordinamento artificiale ed è la scelta più semplice ed efficace.

4.2. Modelli sviluppati

Per affrontare il problema sono state scelte famiglie di modelli molto diverse tra loro, in modo da confrontare approcci con logiche differenti e capire quale si adatta meglio ai dati. Sono stati inclusi modelli lineari semplici (come baseline di riferimento), modelli ad albero singolo, modelli ensemble (che combinano molti alberi), una rete neurale e infine uno strumento di AutoML. Confrontare modelli così diversi permette di verificare se il problema richiede modelli complessi o se anche approcci semplici sono sufficienti. In totale sono stati sviluppati e confrontati **10 modelli**:

1. **Linear Regression** — baseline lineare, nessuna regolarizzazione
2. **Ridge** — regolarizzazione L2, riduce i coefficienti verso zero
3. **Lasso** — regolarizzazione L1, può azzerare feature irrilevanti
4. **Decision Tree** — albero singolo, alto rischio overfitting
5. **Random Forest** — ensemble bagging, riduce la varianza
6. **Gradient Boosting** — boosting sequenziale, riduce bias e varianza
7. **Random Forest Tuned** — ottimizzato con `RandomizedSearchCV`
8. **Gradient Boosting Tuned** — ottimizzato con `RandomizedSearchCV`
9. **Deep Learning Keras** — rete neurale densa 128-64-32-1
10. **AutoML (FLAML)** — selezione automatica del modello e degli iperparametri

4.3. Ottimizzazione iperparametri

È stato usato `RandomizedSearchCV` con 5-fold cross-validation (`n_iter=20`, `random_state=42`) sui due modelli migliori della baseline. La cross-validation opera esclusivamente su `X_train_s`; il test set non è mai coinvolto nella selezione degli iperparametri.

Perché `RandomizedSearch` e non `GridSearch`: lo spazio di possibili combinazioni di iperparametri è molto grande (per il Random Forest oltre 200 combinazioni). Provarle tutte (`GridSearch`) richiederebbe troppo tempo di calcolo. `RandomizedSearch` ne prova invece un campione casuale (20 combinazioni), ottenendo risultati molto vicini all'ottimo in una frazione del tempo. È un buon compromesso tra qualità della ricerca e velocità.

Perché la cross-validation a 5 fold: invece di valutare ogni combinazione di iperparametri su un singolo split, la 5-fold CV divide il training set in 5 parti e addestra/valuta 5 volte, ruotando la parte usata per la validazione. Il risultato è una stima più stabile e affidabile, meno dipendente dal particolare sottoinsieme di dati.

4.4. AutoML con FLAML

Come strumento complementare ai modelli custom, è stato impiegato **FLAML**, la libreria AutoML integrata in Fabric. Con un budget temporale di 120 secondi e seed fisso, FLAML ha esplorato automaticamente diversi algoritmi selezionando **LightGBM** come modello migliore ($\text{RMSE} = 2.138$, $R^2 = 0.947$). Il risultato si colloca al livello del Gradient Boosting base, confermando che i modelli ensemble ottimizzati manualmente (Tuned) restano superiori.

Table 6: Migliori iperparametri trovati

Modello	Parametro	Valore
RandomForest Tuned	<code>n_estimators</code>	200
	<code>max_depth</code>	30
	<code>max_features</code>	log2
	<code>min_samples_split</code>	2
	<code>min_samples_leaf</code>	1
GradientBoosting Tuned	<code>n_estimators</code>	300
	<code>learning_rate</code>	0.1
	<code>max_depth</code>	5
	<code>subsample</code>	0.8
	<code>min_samples_split</code>	2

4.5. Deep Learning con Keras

La rete neurale è stata progettata con architettura “piramidale” ($128 \rightarrow 64 \rightarrow 32 \rightarrow 1$): il numero di neuroni si riduce progressivamente a ogni livello. L’idea è che i primi layer catturano molti dettagli grezzi dei dati, mentre i layer successivi li combinano in rappresentazioni via via più sintetiche, fino alla previsione finale. La struttura prevede:

- Attivazione **ReLU** nei layer nascosti (cattura non linearità)
- **Dropout 0.2** dopo ogni layer nascosto (riduce overfitting)
- Output **lineare** (appropriato per regressione continua)
- Ottimizzatore **Adam** con learning rate 0.001
- **EarlyStopping** con patience=10 (ripristina i pesi migliori)

5. Analisi delle metriche e scelta del modello migliore

5.1. Classifica finale — tutti i modelli

La tabella seguente riporta le metriche di tutti i 10 modelli sul test set, ordinate per RMSE crescente. I modelli ensemble ottimizzati occupano le prime posizioni, seguiti dai modelli base e da AutoML; i modelli lineari chiudono la classifica.

5.2. Modello scelto

Il modello selezionato è il **Random Forest Tuned** con:

- **RMSE = 1.627 anni** — errore medio di previsione
- **$R^2 = 0.970$** — spiega il 97.0% della varianza del target
- **R^2 aggiustato = 0.969** — penalizzato per le 19 feature

Table 7: Metriche comparative di tutti i modelli sul test set (586 righe)

Modello	RMSE	MAE	R^2	Adj R^2	MSE
RandomForest Tuned	1.627	1.079	0.970	0.969	2.645
GradientBoosting Tuned	1.640	1.090	0.969	0.968	2.689
RandomForest	1.679	1.100	0.968	0.967	2.821
GradientBoosting	2.131	1.560	0.948	0.946	4.542
AutoML (LightGBM)	2.138	1.565	0.947	0.946	4.571
DecisionTree	2.668	1.642	0.918	0.916	7.120
DeepLearning Keras	3.300	2.466	0.875	0.871	10.893
LinearRegression	3.877	2.832	0.827	0.822	15.033
Ridge	3.883	2.835	0.827	0.821	15.077
Lasso	4.366	3.151	0.781	0.774	19.059

Nota metodologica

R^2 aggiustato: è una versione dell' R^2 che tiene conto del numero di feature usate, penalizzando i modelli con molte variabili poco utili. Nel nostro caso la differenza con l' R^2 normale è minima (0.970 vs 0.969), perché il numero di osservazioni è molto maggiore del numero di feature.

5.3. Interpretazione per famiglia di modelli

- **Ensemble ($R^2 > 0.94$):** dominano grazie alla capacità di catturare relazioni non lineari tra le feature sanitarie ed economiche
- **AutoML ($R^2 = 0.947$):** LightGBM raggiunge il livello del Gradient Boosting base ma non supera i modelli ensemble ottimizzati manualmente
- **Decision Tree ($R^2 = 0.918$):** performance intermedia con evidenti segnali di overfitting ($R^2_{\text{train}} \approx 1.0$)
- **Keras DL ($R^2 = 0.875$):** discreto ma inferiore agli ensemble. Con circa 2300 campioni tabulari il vantaggio delle reti neurali non si manifesta pienamente
- **Lineari ($R^2 < 0.83$):** il gap con gli ensemble conferma la presenza di relazioni non lineari significative nel dataset

5.4. Feature Importance

Le variabili più predittive nel modello migliore sono indicatori sanitari diretti. `hiv_aids` e `adult_mortality` insieme spiegano oltre il 40% dell'importanza totale.

5.5. Analisi dei residui

I residui del modello migliore mostrano:

- **Media residui ≈ 0** — nessun bias sistematico
- **Distribuzione approssimativamente normale** — assenza di pattern sistematici

Table 8: Feature Importance — modello migliore (valori indicativi)

Feature	Importanza	Interpretazione
hiv_aids	23.6%	Mortalità per HIV/AIDS per 1000 nati
adult_mortality	20.2%	Mortalità adulta per 1000 abitanti
income_composition_of_resources	15.9%	Indice di sviluppo umano composito
schooling	10.6%	Anni di istruzione
bmi	7.4%	Indicatore condizioni nutrizionali
Altre 14 feature	22.3%	—

- **Scatter omogeneo** nel grafico residui vs predetti — omoschedasticità

5.6. Analisi overfitting

Table 9: Confronto R^2 train vs test — analisi overfitting

Modello	R^2 _train	R^2 _test	Gap	Esito
LinearRegression	0.824	0.827	-0.003	OK
Ridge	0.824	0.827	-0.003	OK
Lasso	0.781	0.781	0.000	OK
RandomForest	0.992	0.968	+0.024	OK
RandomForest Tuned	0.990	0.970	+0.020	OK
GradientBoosting	0.965	0.948	+0.017	OK
GradientBoosting Tuned	0.984	0.969	+0.015	OK
AutoML (LightGBM)	0.961	0.947	+0.014	OK
DecisionTree	1.000	0.918	+0.082	OVERFITTING
DeepLearning Keras	0.896	0.875	+0.021	OK

Il Decision Tree presenta overfitting (gap = 0.082): senza regolarizzazione, memorizza il training set. Gli ensemble gestiscono l'overfitting grazie al bagging (RF) e alla regolarizzazione implicita del boosting (GB). I modelli lineari mostrano un gap leggermente negativo (-0.003): performano marginalmente meglio sul test che sul train, un fenomeno normale dovuto alla variabilità campionaria quando il modello è semplice e non soffre di overfitting.

Nota metodologica

Feature causalmente rilevanti: le variabili `adult_mortality`, `infant_deaths` e `under_five_deaths` sono cause dirette dell'aspettativa di vita, raccolte dall'OMS in modo indipendente. La loro alta importanza nel modello riflette quindi relazioni reali tra i dati e non un problema di costruzione del dataset. I valori estremi di queste variabili corrispondono a paesi con condizioni sanitarie critiche (ad esempio l'epidemia di HIV in Africa sub-sahariana negli anni 2000–2010) e contengono informazione fondamentale per le previsioni.

6. Interpretazione dei risultati e conclusioni

6.1. Principali risultati

- L'aspettativa di vita è fortemente determinata da indicatori sanitari diretti (`hiv_aids`, `adult_mortality`) più che da indicatori economici generali (`gdp`, `status_encoded`)
- I modelli ensemble superano significativamente i modelli lineari (R^2 0.97 vs 0.83), confermando la presenza di relazioni non lineari nel dataset
- La rimozione degli outlier IQR, inizialmente applicata seguendo le indicazioni della consegna, si è rivelata controproducente: eliminava dati reali (epidemie, paesi ad alta mortalità) degradando le metriche di tutti i modelli. La pipeline finale non applica rimozione outlier
- Il tuning degli iperparametri ha migliorato le performance degli ensemble, portando il Random Forest Tuned a $R^2 = 0.970$
- AutoML (LightGBM) ha raggiunto il livello del Gradient Boosting base ma non ha superato i modelli ensemble ottimizzati manualmente, confermando la qualità dell'approccio custom
- Il Deep Learning Keras ($R^2 = 0.875$) performa peggio degli ensemble su questo dataset tabulare strutturato di dimensioni moderate

6.2. Limitazioni

- **Dati aggregati per paese e anno:** non catturano la variabilità intra-nazionale (differenze tra regioni o gruppi demografici)
- **Range temporale 2000–2015:** il modello potrebbe non generalizzare bene su dati post-2015
- **Multicollinearità:** `thinness_1_19_years` e `thinness_5_9_years` sono quasi identiche ($r=0.93$). In un contesto produttivo si rimuoverebbe una delle due
- **Architettura Keras:** con circa 2300 campioni e 19 feature tabulari, architetture più semplici o diversamente configurate potrebbero produrre risultati leggermente diversi
- **Esclusione country:** variabili latenti specifiche per paese potrebbero migliorare le performance; alternative come l'aggregazione per continente o area OMS potrebbero essere esplorate

6.3. Possibili sviluppi futuri

- Testare XGBoost come alternativa al Gradient Boosting nativo
- Includere feature temporali per catturare trend annuali per paese
- Sperimentare la trasformazione logaritmica su `hiv_aids`, `measles`, `under_five_deaths` invece della rimozione outlier
- Utilizzare un validation set separato per la scelta finale del modello, mantenendo il test set unicamente per la valutazione conclusiva

7. Divisione del lavoro tra i membri del gruppo

Table 10: Divisione del lavoro nel gruppo CereaMorlotti

Membro	Ruolo	Attività principali
Morlotti Andrea	Data Scientist B	EDA regressione, pipeline Power Query cleaning, sviluppo e tuning modelli regressione, MLflow tracking, dashboard Power BI regressione (EDA + ML), report tecnico
Cerea Gianluca	Data Scientist A	EDA classificazione, pipeline Power Query cleaning, sviluppo e tuning modelli classificazione, MLflow tracking, dashboard Power BI classificazione

8. Link al Workspace Fabric

Tutti gli asset del progetto sono accessibili nel workspace Microsoft Fabric **Progetto-Valtellina** del gruppo CereaMorlotti, all'interno del Lakehouse **StartingStorage**. Il workspace contiene i notebook di EDA e modellazione, il Dataflow di cleaning, l'esperimento e i modelli registrati su MLflow, e le due dashboard Power BI (EDA e risultati ML).